Configuración del sistema

Referencia de todos los ajustes del sistema: qué se configura, dónde vive cada valor y qué pasa si se cambia. El **porqué** de cada decisión está en el capítulo 1; el **procedimiento** de una jornada, en el capítulo 3.

Versión 1.3 — 23 de octubre de 2026. La velocidad de avance la mide un GPS USB (VK-162) conectado a la Pi, con la antena en el techo de la cabina; sale el nodo `NH02` y quedan siete ESP32 (§1, §2.2, §3, §4.4, §7). La versión 1.2 pasó los nodos de sensor a ESP32-C6 con ultrasónico RCWL-1655, los nodos a 4 pilas AA Ni-MH sin microSD, la central a una batería LiFePO4 de 12 V y el punto de acceso a un router Tenda N301. La revisión del 24 de octubre corrige la unidad del servicio (§3.3) y añade la primera prueba del GPS (§3.4) y el acceso en mesa sin el punto de acceso (§3.5).

---

1 · Dónde vive cada cosa

La regla es que **cada valor vive en un solo sitio**, y que ese sitio sea el que menos cueste cambiar.

Qué	Dónde	Cambiarlo cuesta
Censo de nodos, planes, ranuras, ROI	`config/censo.py`	Editar y reiniciar la central
Constantes físicas medidas con metro	`config/calibracion.json`	Editar y reiniciar
Receptor GPS y su filtro	`config/censo.py` (`GPS`) y `RICEVI_GPS_PUERTO`	Editar y reiniciar
Parámetros del servidor	`central/app/config.py` o variables de entorno	Reiniciar el servicio
Identidad y credenciales de un nodo	NVS de la placa	Conectar por USB una vez
Red del punto de acceso	El router	Su interfaz web
Pesos de los modelos	`modelos/pesos/`	Copiar archivos y reiniciar

Lo que NO está en el firmware: ritmo de captura, resolución, calidad JPEG, ROI, ranura de reloj, pulsos por vuelta, offset del ultrasónico, umbrales de batería y ahorro de radio. Todo eso llega en el plan que la central envía. Si estuviera compilado, cambiar el ritmo de una cámara sería desmontar una caja del cabezal de una cosechadora. Siete veces.

---

2 · El punto de acceso

2.1 Ajustes obligatorios

Parámetro	Valor	Si se pone mal
SSID	`RiceVi-M01`	—
Seguridad	WPA2-PSK	WPA3 puro puede no negociar con el ESP32
Banda	**Solo 2,4 GHz**	El nodo no ve la red, y parece una placa averiada
Canal	**Fijo: 1, 6 u 11**	Un cambio de canal en marcha

		desconecta a los siete a la vez
Ancho de canal	20 MHz	40 MHz añade interferencia sin beneficio
Aislamiento de clientes	**DESACTIVADO**	Los nodos ven Internet pero no ven la Pi. **Fallo número uno**
Band steering	Desactivado	Confunde a clientes simples
802.11k / v / r	Desactivado	Itinerancia que un ESP32 no necesita
Portal cautivo	Ninguno	Un ESP32 no puede aceptar términos en una página
Ahorro de energía del AP	Desactivado	Retrasa las tramas donde está el cuello de botella

Con el Tenda N301: solo acepta reservas DHCP **dentro** del rango de su servidor DHCP. El rango va de `.2` a `.99`, y las reservas se crean en cuanto se conoce cada MAC (detalle en `config/red/router.md`).

2.2 Direccionamiento

Equipo	IP	Cómo se fija
`AP01`	`192.168.40.1`	Configuración del router
`RPI01`	`192.168.40.2`	**Reserva DHCP por MAC**
`CAM01`–`CAM04`	`.11` – `.14`	Reserva DHCP
`NH01`	`.21`	Reserva DHCP. La `.22` queda libre: era de `NH02`
`NU01`, `NU02`	`.23`, `.24`	Reserva DHCP
`HMI01`	`.30`	Reserva DHCP
Diagnóstico	`.40` – `.49`	Rango libre

Reserva DHCP y no IP estática en el equipo. Con la IP grabada en el nodo, llevar una placa al taller obliga a reconfigurarla. Con reserva, el nodo pide dirección como cualquier cliente y en la red de campo siempre recibe la misma.

Las MAC se anotan al provisionar cada placa y van a `config/censo.py`. `python config/censo.py` avisa mientras falte alguna.

---

3 · La central

3.1 Instalación

```
./herramientas/copiar_a_pi.ps1
ssh ricevi-rasp@ricevi-rasp-01.local "~/config-ricevi-campo/herramientas/instalar_pi.sh"
```

`instalar_pi.sh` es idempotente y **comprueba lo que no se debe dar por hecho**:

Comprueba	Por qué
Que los datos estén en el NVMe y no en la microSD	17 GB por jornada no caben en una microSD, ni aguanta la escritura
Que haya batería de RTC	De ella depende que la hora de los datos sea fiable sin Internet

Que exista control de ventilador	Sin él la CPU se estrangula bajo carga
Que el PMIC no reporte subtensión	Es la señal temprana de un problema eléctrico
Que el GPS se vea en `/dev/serial/by-id/`, que el usuario esté en `dialout` y que ModemManager o `gpsd` no le quiten el puerto	Sin el receptor no hay velocidad de avance ni *reel index*, y el síntoma sería un HMI con la velocidad en blanco

3.2 Variables de entorno

Se ajustan en la unidad del servicio, sin editar código:

Variable	Por defecto	Qué controla
`RICEVI_MODELO`	`ninguno`	`ninguno` = fase de dataset · `ncnn` = fase de inferencia
`RICEVI_DIR_DATOS`	`/datos/ricevi`	**Debe apuntar al NVMe**
`RICEVI_EXIGIR_TOKEN`	`1`	Autenticación de nodos. En campo, siempre `1`
`RICEVI_COLA_MAX`	`8`	Profundidad de la cola de inferencia
`RICEVI_HILOS_INF`	`1`	Hilos de inferencia
`RICEVI_VIGENCIA_MS`	`4000`	Edad máxima de un frame para que merezca inferirse
`RICEVI_TURNOS`	`1`	Subidas simultáneas permitidas
`RICEVI_RESERVA_MB`	`8192`	Por debajo, deja de aceptar frames
`RICEVI_GPS`	`1`	Lee el GPS. Con `0` arranca sin él (banco): no hay velocidad ni *reel index*
`RICEVI_GPS_PUERTO`	`auto`	`auto` busca un u-blox en `/dev/serial/by-id/`. Se fija una ruta solo si hay dos receptores
`RICEVI_GPIO_CONTACTO`	*(vacío)*	**Sin uso.** La central ya no se conecta a la máquina: lleva batería propia y el apagado es manual (capítulo 1, §3.5)

Para pasar a inferencia:

```
sudo systemctl edit ricevi-central     #  Environment=RICEVI_MODELO=ncnn
sudo systemctl restart ricevi-central
```

3.3 El servicio

`Restart=always` y no `on-failure`: si el proceso termina por cualquier motivo durante una jornada, hay que volver a capturar. `StartLimitIntervalSec=0` evita que systemd se rinda tras cinco intentos y deje la jornada perdida en silencio. Va en la sección `[Unit]`, porque en `[Service]` systemd lo ignora.

`TimeoutStopSec=40` da margen para que la central cierre la base y consolide el disco antes de que systemd la mate.

3.4 El GPS

El receptor VK-162 (u-blox 7) va a un USB de la Pi, con la antena en el techo de la cabina. La central lo lee directamente, sin `gpsd`, y guarda una solución por segundo como telemetría de `RPI01`/`GPS01`. Sus ajustes viven en `config/censo.py`:

Ajuste	Valor	Qué controla
`puerto`	`auto`	Busca `/dev/serial/by-id/*u-blox*`. **Nunca** toma `/dev/ttyACM0` a ciegas: la ESP32-C6 también aparece así al provisionarla desde la Pi
`ventana_filtro_s`	3,0	Media móvil de la velocidad que usa el *reel index*
`umbral_reposo_mps`	0,2	Por debajo, la filtrada vale 0 con código `reposo`: con la máquina quieta el receptor sigue dando ruido
`hdop_max`	5,0	Por encima, la muestra entra como `dudosa` (`hdop_alto`)
`incertidumbre_ms`	100	Retraso declarado de la trama respecto al instante de la solución

La posición no se guarda. La central lee la latitud y la longitud solo para validar la trama y las descarta.

Para comprobar el receptor a mano, con el servicio parado (si no, los dos programas se reparten las tramas):

```
ls -l /dev/serial/by-id/          # debe aparecer usb-u-blox_..._GPS_GNSS_Receiver
sudo systemctl stop ricevi-central
timeout 5 cat /dev/serial/by-id/*u-blox*   # tramas $GPRMC, $GPGGA...
sudo systemctl start ricevi-central
curl -s localhost:8000/api/status | python3 -m json.tool | grep -A14 '"gps"'
```

En `$GPRMC` la tercera columna es `A` cuando hay solución y `V` cuando no.

Primera prueba con el receptor (24-oct-2026). Conectado a un USB de la Pi, la central lo encontró por `by-id` sin ningún ajuste (ModemManager y `gpsd` inactivos, usuario en `dialout`) y dio solución al aire libre desde el primer minuto, una por segundo. Vio solo 5 satélites con HDOP 2,6, que pasaron a 6 y 1,9 en unos diez minutos. Sin mover la antena a propósito, la velocidad cruda tuvo una media de 0,07 m/s y picos de 0,47 m/s, y 4 de 311 muestras filtradas pasaron el umbral de reposo, con 0,22–0,30 m/s. Al mover la antena a mano, la central marcó tramos de unos 8 s con picos de 0,86 m/s.

Cinco o seis satélites es poco para cielo abierto. Antes de cambiar `umbral_reposo_mps` o `ventana_filtro_s` hay que repetir la medida con la antena plana, mirando al cielo y a un metro o más de la Pi y del SSD, y ver si con ocho o más satélites el ruido en reposo baja del umbral.

Para seguir el GPS sin parar el servicio, desde la propia Pi:

```
watch -n1 'curl -s localhost:8000/api/status | python3 -m json.tool | grep -A16 "\"gps\""'
sqlite3 -readonly -column /datos/ricevi/ricevi.db "SELECT seq, magnitud, round(valor,2) valor, calidad,
codigo_calidad FROM telemetria WHERE sensor_id='GPS01' ORDER BY id DESC LIMIT 8"
```

3.5 Acceso en mesa, sin el punto de acceso

En campo, el HMI está en `http://192.168.40.2:8000`, dentro de la red del router. En mesa, sin el router, la Pi se conecta a cualquier Wi-Fi que tenga guardada y la central funciona igual, con el GPS incluido. Lo único que falta son los nodos, porque solo conocen la red del router. El HMI no pide clave y se abre de tres maneras:

Desde	Dirección	Requisito
La propia Pi	`http://localhost:8000`	Monitor o pantalla compartida de Raspberry Pi Connect
Un portátil en la misma Wi-Fi	`http://ricevi-rasp-01.local:8000`	mDNS (Avahi), que deja instalado `instalar_pi.sh`

Un portátil en cualquier red	`http://ricevi-rasp-01:8000`	La Pi y el portátil en la misma cuenta de Tailscale, los dos con Internet

Sin Internet ni Wi-Fi conocida, el hotspot de un teléfono sirve de red común. Tailscale es solo acceso de mantenimiento: en el lote no hay Internet y la captura no depende de él.

Desde PowerShell, sin entrar a la Pi, una línea por segundo con el estado del GPS:

```
while ($true) { $g = (Invoke-RestMethod http://ricevi-rasp-01:8000/api/status).gps.ultima; "{0:HH:mm:ss}  vel
{1:N2} m/s  filtrada {2:N2} m/s  {3}  sat {4}  hdop {5}" -f (Get-Date), $g.ground_speed_mps,
$g.ground_speed_filtrada_mps, $g.codigo, $g.satelites, $g.hdop; Start-Sleep 1 }
```

Para no escribir la contraseña en cada `ssh` ni en cada paso de `copiar_a_pi.ps1`, se autoriza una vez la llave del portátil:

```
scp $env:USERPROFILE\.ssh\id_ed25519.pub ricevi-rasp@ricevi-rasp-01.local:/tmp/llave.pub
ssh ricevi-rasp@ricevi-rasp-01.local "mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat /tmp/llave.pub >>
~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys && rm /tmp/llave.pub"
```

No conviene pasar la llave por una tubería (`Get-Content llave.pub | ssh ...`). Windows PowerShell 5.1 puede anteponerle una marca BOM al texto, la línea queda ilegible para `sshd` y la llave se ignora sin ningún mensaje de error. `scp` copia el archivo tal cual. `sudo` sigue pidiendo la contraseña, así que instalar la unidad del servicio o reiniciarlo se hace con `ssh -t`.

---

4 · Los nodos

4.1 Un binario por rol, no por nodo

Se compilan **tres** binarios —cámara, Hall, ultrasónico— y la identidad se graba en NVS. Las cuatro cámaras corren el mismo código.

Con un binario por nodo, copiar el equivocado convierte a `CAM03` en `CAM02`, y el síntoma sería un `409` que nadie sabría explicar, o peor: imágenes atribuidas a la vista equivocada.

4.2 Qué se graba en NVS

Clave	Contenido
`node_id`	`CAM01`…`NU02`. **Sin ella el nodo no arranca**: LED rojo fijo y silencio
`token`	Credencial del nodo
`wifi_ssid`, `wifi_pass`	Red
`central_ip`	Primera vía de descubrimiento

```
python herramientas/generar_tokens.py
python herramientas/provisionar_nodo.py --node-id CAM01 --puerto COM7 --pass "<clave>"
```

El script imprime la **MAC**: hay que copiarla a `config/censo.py` para poder crear la reserva DHCP. Y hay que **pegar la etiqueta en la caja antes de montarla**: siete placas iguales dentro de siete cajas atornilladas son indistinguibles.

Provisionando desde la Pi con el GPS conectado, `/dev/ttyACM0` puede ser el GPS y no la ESP32-C6. Hay que elegir el puerto de la placa por su nombre en `/dev/serial/by-id/` (la C6 aparece con el conversor CH343, la cámara con el CH340C), o desconectar el GPS mientras se provisiona.

4.3 Ajustes de compilación

Cámaras (GOOUUU ESP32-S3-CAM):

```
Placa              ESP32S3 Dev Module
PSRAM              OPI PSRAM          <- imprescindible en las cámaras
Flash Size         16MB (128Mb)
Partition Scheme   16M Flash (3MB APP/9.9MB FATFS)
USB CDC On Boot    Disabled           <- programando por el USB-C "TTL" (CH340C)
```

Sin `OPI PSRAM` la cámara no arranca, y el error no menciona la PSRAM por ninguna parte.

Nodos de sensor (ESP32-C6 DevKitC):

```
Placa              ESP32C6 Dev Module
Partition Scheme   Huge APP           <- con la de por defecto ocupan el 92-93 %
```

Los pines de la C6 los elige el propio código con `CONFIG_IDF_TARGET_ESP32C6`: Hall en GPIO10; ultrasónico con Trig en GPIO11 y Echo en GPIO10; batería en GPIO1; LED de estado en GPIO8.

4.4 El plan que recibe cada nodo

Nodo	Captura	Resolución	Infiere	Ahorro radio
`CAM01`	1,0 img/s	1280×720	1 de cada 2	No
`CAM02` `CAM03`	1,0 img/s	1280×720	1 de cada 1	No
`CAM04`	0,5 img/s	1280×720	1 de cada 3	No
`NH01`	10 Hz	—	—	**Sí**
`NU01` `NU02`	8 Hz	—	—	**Sí**

`CAM04` es la única con **exposición, ganancia y balance de blancos manuales**. Dentro de la tolva, si el control automático ajusta la imagen entre frames, el porcentaje segmentado cambia por razones que no tienen que ver con el material y las tolvadas dejan de ser comparables.

`CAM04` recibirá además, en el mismo plan, la **ventana de encendido del LED**: 150–300 ms alrededor de cada foto. **Pendiente:** `camara.ino` todavía no maneja el LED. El nodo enciende el LED, descarta uno o dos cuadros, captura y apaga. Esos milisegundos son el único ajuste del LED que existe, y como todo lo demás llega en el plan, cambiarlos es editar un archivo y reiniciar la central, no abrir la tolva.

4.5 Código de colores del LED

Es el único diagnóstico con el nodo dentro de una caja cerrada. Conviene pegar esta tabla en la cabina:

Color	Significado
Rojo fijo	**Sin identidad en NVS**, o batería crítica
Rojo parpadeando	Sin Wi-Fi
Ámbar parpadeando	Con Wi-Fi, sin encontrar la central
Verde parpadeando	Registrado y operando
Verde doble	Subiendo un frame
Azul	Sincronizando reloj
Ámbar tras una captura	No se pudo subir el frame y, sin microSD, se perdió

Morado	Pendientes en la microSD por reenviar (solo si se activa `RICEVI_SD_HABILITADA`; hoy los nodos no llevan microSD)
Naranja	Batería por debajo del umbral de aviso

5 · Baterías

Parámetro	Valor
Configuración de los nodos	4 × AA Ni-MH **en serie** (4S), directo al pin de 5 V, con fusible de 1 A e interruptor
`CAM04`	Igual que los demás. El LED de la tolva lleva su propia batería de 9 V (capítulo 1, §3.6)
Central	**Batería LiFePO4 de 12,8 V y 12 Ah**, compartida con `AP01` (capítulo 1, §3.5)
Capacidad	2600 mAh · 4,8 V ≈ 12,5 Wh (la caja dice 3600 mAh: **a medir**)
Descarga completa	4,0 V (1,0 V por pila)
Aviso (firmware)	4,50 V
Crítico (firmware)	4,35 V
Medida	Divisor 150 kΩ / 100 kΩ (factor 2,5) al GPIO1. **Aún sin montar**: hasta entonces la batería llega nula
Reporte	En el latido, cada 10 s

Al llegar al umbral **crítico**, un nodo de cámara **deja de capturar ordenadamente** en vez de morirse a mitad de una subida y dejar un archivo a medias.

No se carga en la máquina. Las pilas se cargan fuera, con su cargador, y nunca en caliente. Hoy hay un juego por nodo; con un segundo juego se podrían trabajar dos jornadas seguidas sin esperar cargas.

La batería de la central se carga con su cargador LiFePO4 de 14,6 V: con 2 A, una noche.

El HMI muestra la tensión de los siete nodos, y `verificar_campo.py` se niega a dar el visto bueno si alguno sale por debajo del umbral de aviso. **Mientras el divisor no esté montado la tensión llega nula**, así que las pilas se comprueban con el multímetro antes de salir. La batería de la central no tiene medida propia: se comprueba antes de salir y se cambia con la central apagada.

6 · Modelos

```
modelos/pesos/
  manifiesto.json          obligatorio: sin él no se carga nada
  post_corte.param  .bin
  tolva.param       .bin
  acame.param       .bin
```

El manifiesto lleva versión, SHA-256 de cada `.bin`, tamaño de entrada, clases, clase objetivo y **clase a ignorar**. Los píxeles ignorados no entran en el denominador del porcentaje: meterlos lo diluiría de forma distinta en cada imagen, según cuánta máquina o cielo se colara en el encuadre.

Si el modelo no carga —faltan pesos, falta la biblioteca, el hash no cuadra— la central vuelve sola a la fase de dataset y lo registra como evento. Perder el diagnóstico es aceptable; perder la jornada de captura no.

Despliegue en sombra

Antes de promover un modelo se ejecuta sobre los mismos frames que el vigente, sin mostrarse al operario, y se comparan las salidas durante al menos una jornada. Hay que contarlo en el presupuesto de CPU: durante la prueba el consumo de inferencia se duplica.

---

7 · Verificación

Script	Dónde corre	Qué comprueba
`config/censo.py`	Cualquiera	Coherencia del censo y calibraciones sin medir
`herramientas/prueba_humo.py`	Portátil	52 comprobaciones de la central, **sin hardware**
`herramientas/verificar_campo.py`	Portátil	El montaje completo, antes de arrancar el motor

`prueba_humo.py` se ejecuta tras cualquier cambio en `central/app/`. Levanta la central en memoria y recorre el camino entero: registro, reloj, turnos, frames, idempotencia, telemetría, la regla del nulo, el GPS con tramas NMEA escritas a mano (conversión, suma de verificación, sin solución, reposo y fusión), batería, trazabilidad y que cada reinicio abra una sesión nueva aunque la anterior no guardara frames.

Está pensado para el portátil. En la Pi solo pasa entero con `TMPDIR` en el NVMe, porque `/tmp` es un disco en RAM de 4 GB y la central rechaza frames con menos de `RESERVA_MB` (8 GB) libres, y con `httpx` instalado aparte del `.venv`.

`verificar_campo.py` comprueba doce cosas en orden, de modo que **la primera que falla dice dónde está el problema**: censo, tokens, central, nodos en línea, baterías, relojes, producción de cada cámara y sensor, calidad de las lecturas, GPS con solución, disco, temperatura y calibraciones.

Las once primeras son **bloqueantes**. La última es un aviso: se puede capturar dataset con calibraciones provisionales, pero las magnitudes derivadas no se podrán publicar. El GPS sin solución es bloqueante porque sin él no hay velocidad en toda la jornada: la comprobación se hace al aire libre.